

```
In [1]: import pandas as pd
import numpy as np

import matplotlib as mpl
import matplotlib.pyplot as plt

import warnings
warnings.filterwarnings(action = 'ignore')

from IPython.display import Image

for i in [pd, np, mpl]:
    print(i.__name__, i.__version__)
```

```
pandas 2.1.4
numpy 1.26.4
matplotlib 3.8.0
```

```
In [2]: Image('7.png')
```

Out[2]: **1-2. 결측치, 이상치 처리**

0. 데이터셋 소개

Titanic

[Titanic](#) 탑승객의 생존 유무를 담은 데이터셋입니다.

Data	Description	Dictionary
PassengerId	Passenger Id, Index	
Survived	Survival	0 = No, 1 = Yes
Pclass	Ticket class	1 = 1st, 2 = 2nd, 3 = 3rd
Name	Name	
Sex	Sex	
Age	Age in years	
Sibsp	# of siblings / spouses aboard the Titanic	
Parch	# of parents / children aboard the Titanic	
Ticket	Ticket number	
Fare	Passenger fare	
Cabin	Cabin number	
Embarked	Port of Embarkation	C = Cherbourg, Q = Queenstown, S = Southampton

다양한 형식의 데이터를 가지고 있으며, 여러 가지 아이디어를 생각하고 시도할 만한 요소가 많게끔 기획된 데이터셋입니다.

```
In [5]: df_titanic = pd.read_csv('data/titanic.csv', index_col='PassengerId')
df_titanic.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 891 entries, 1 to 891
Data columns (total 11 columns):
 #   Column      Non-Null Count  Dtype
---  ---
 0   Survived    891 non-null    int64
 1   Pclass      891 non-null    int64
 2   Name        891 non-null    object
 3   Sex         891 non-null    object
 4   Age         714 non-null    float64
 5   SibSp       891 non-null    int64
 6   Parch       891 non-null    int64
 7   Ticket      891 non-null    object
 8   Fare        891 non-null    float64
 9   Cabin       204 non-null    object
10   Embarked    889 non-null    object
dtypes: float64(2), int64(4), object(5)
memory usage: 83.5+ KB
```

```
In [6]: Image('8.png')
```

Out[6]: **Wild blueberry**

[Wild blueberry](#) 야생 블루베리의 수확량을 시뮬레이션한 데이터셋입니다.

Features	Unit	Description
Row#		Index
Clonesize	m2	The average blueberry clone size in the field
Honeybee	bees/m2/min	Honeybee density in the field
Bumbles	bees/m2/min	Bumblebee density in the field
Andrena	bees/m2/min	Andrena bee density in the field
Osmia	bees/m2/min	Osmia bee density in the field
MaxOfUpperTRange	°C	The highest record of the upper band daily air temperature during the bloom season
MinOfUpperTRange	°C	The lowest record of the upper band daily air temperature
AverageOfUpperTRange	°C	The average of the upper band daily air temperature
MaxOfLowerTRange	°C	The highest record of the lower band daily air temperature
MinOfLowerTRange	°C	The lowest record of the lower band daily air temperature
AverageOfLowerTRange	°C	The average of the lower band daily air temperature
RainingDays	Day	The total number of days during the bloom season, each of which has precipitation larger than zero
AverageRainingDays	Day	The average of raining days of the entire bloom season
fruitset		The amount of fruit per tree
fruitmass		The mass of a fruit
seeds		The mass of a seed
yield		The amount of yield

```
In [2]: df_berry = pd.read_csv('data/wild_blueberry.csv', index_col='Row#')
df_berry.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 777 entries, 0 to 776
Data columns (total 17 columns):
#   Column                Non-Null Count  Dtype
---  -
0   clonesize              777 non-null    float64
1   honeybee               777 non-null    float64
2   bumbles                777 non-null    float64
3   andrena                777 non-null    float64
4   osmia                  777 non-null    float64
5   MaxOfUpperTRange       777 non-null    float64
6   MinOfUpperTRange       777 non-null    float64
7   AverageOfUpperTRange   777 non-null    float64
8   MaxOfLowerTRange       777 non-null    float64
9   MinOfLowerTRange       777 non-null    float64
10  AverageOfLowerTRange    777 non-null    float64
11  RainingDays             777 non-null    float64
12  AverageRainingDays      777 non-null    float64
13  fruitset                777 non-null    float64
14  fruitmass               777 non-null    float64
15  seeds                   777 non-null    float64
16  yield                   777 non-null    float64
dtypes: float64(17)
memory usage: 109.3 KB
```

1. 결측치 처리

결측 레코드 제거

결측이 있는 데이터들을 제거합니다.

- 처리는 쉽지만 정보의 손실이 있습니다.

```
In [8]: # 결측이 있으면 제거합니다.
# 실제로 반영을 하려면 inplace=True를 dropna에 전달하거나
# df_titanic = df_titanic.dropna() 처럼 대입을 시킵니다.
df_titanic.dropna()
```

Out[8]:

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C
4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S
7	0	1	McCarthy, Mr. Timothy J	male	54.0	0	0	17463	51.8625	E46	S
11	1	3	Sandstrom, Miss. Marguerite Rut	female	4.0	1	1	PP 9549	16.7000	G6	S
12	1	1	Bonnell, Miss. Elizabeth	female	58.0	0	0	113783	26.5500	C103	S
...	...	...	...	...	...	...	...	...	...	...	...
872	1	1	Beckwith, Mrs. Richard Leonard (Sallie Monypeny)	female	47.0	1	1	11751	52.5542	D35	S
873	0	1	Carlsson, Mr. Frans Olof	male	33.0	0	0	695	5.0000	B51 B53 B55	S
880	1	1	Potter, Mrs. Thomas Jr (Lily Alexenia Wilson)	female	56.0	0	1	11767	83.1583	C50	C
888	1	1	Graham, Miss. Margaret Edith	female	19.0	0	0	112053	30.0000	B42	S
890	1	1	Behr, Mr. Karl Howell	male	26.0	0	0	111369	30.0000	C148	C

183 rows × 11 columns

In [9]: `df_titanic.dropna().isna().sum().to_frame()`

```
Out[9]:
```

Survived	0
Pclass	0
Name	0
Sex	0
Age	0
SibSp	0
Parch	0
Ticket	0
Fare	0
Cabin	0
Embarked	0

```
In [10]: # 결측을 삭제 후 결과를 확인합니다.
df_titanic.dropna().isna().sum().to_frame().T
```

```
Out[10]:
```

	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	0	0	0	0	0	0	0	0	0	0	0

단순 대체

범주형 데이터

1. 결측 자체도 하나의 범주값으로 보고, 결측을 의미하는 범주를 생성하여 대체합니다.

[Ex.1]

Embarked의 결측치를 살펴보고 처리합니다.

```
In [6]: # Embark에 등장하는 범주를 가져와 봅니다.
df_titanic['Embarked'].unique()
```

```
Out[6]: array(['S', 'C', 'Q', nan], dtype=object)
```

```
In [7]: # pd.Series.isna를 통해 결측 유무를 가져올 수 있습니다.
df_titanic.loc[df_titanic['Embarked'].isna()]
```

```
Out[7]:
```

	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
PassengerId											
62	1	1	Icard, Miss. Amelie	female	38.0	0	0	113572	80.0	B28	NaN
830	1	1	Stone, Mrs. George Nelson (Martha Evelyn)	female	62.0	0	0	113572	80.0	B28	NaN

```
In [8]: # pd.Series.fillna를 통해 결측치를 대체할 수 있습니다.
# Unknown 범주를 Unk 약어로 나타냅니다.
df_titanic['Embarked'] = df_titanic['Embarked'].fillna('Unk')
```

```
In [9]: df_titanic['Embarked'].value_counts()
```

```
Out[9]: S      644
C      168
Q       77
Unk       2
Name: Embarked, dtype: int64
```

2. 최빈값으로 대체

[Ex.2]

Cabin은 복수의 요소로 구성이 되어 있고, 공백으로 구분 되어 있습니다.

이 중 맨 처음 나오는 Cabin만 남깁니다.

앞자리 알파벳을 Cabin_Type 그리고,

뒷자리 숫자는 Cabin_No로 나누어 파생 변수를 만듭니다.

Cabin_Type을 Pclass 별로 Cabin_Type의 최빈값으로 대체합니다.

만일 동수가 등장할 경우 사전순으로 앞에 있는 것으로 대체합니다.

```
In [10]: '''
`df_titanic.loc[df_titanic['Cabin'].notna(), 'Cabin'].str.split(' ')`는
Pandas DataFrame인 `df_titanic`의 'Cabin' 열에서
결측치가 아닌 값을 가진 행들을 선택하고,
해당 행들의 'Cabin' 값을 공백을 기준으로 분할하는 과정을 나타냅니다.

- `df_titanic['Cabin'].notna()` :
  'Cabin' 열의 값이 결측치가 아닌지를 나타내는 불리언 시리즈를 반환합니다.
- `df_titanic.loc[]` :
  불리언 인덱싱을 사용하여 조건을 만족하는 행들을 선택합니다.
- `.str.split(' ')` :
  선택된 행들의 'Cabin' 값들을 공백을 기준으로 분할합니다.
  각 값은 공백을 기준으로 분할되어 리스트로 반환됩니다.

따라서 이 코드는 'Cabin' 열에서 결측치가 아닌 값을 가진 행들의
'Cabin' 값들을 공백을 기준으로 분할하여 리스트로 반환합니다.
'''

# 공백으로 나눕니다.
# 873번 데이터의 Cabin에는 여러 개의 객실이 있습니다.
df_titanic.loc[df_titanic['Cabin'].notna(), 'Cabin'].str.split(' ')
```

```
Out[10]: PassengerId
2          [C85]
4          [C123]
7          [E46]
11         [G6]
12        [C103]
...
872         [D35]
873    [B51, B53, B55]
880         [C50]
888         [B42]
890        [C148]
Name: Cabin, Length: 204, dtype: object
```

```
In [11]: '''
코드는 'Cabin' 열에서 결측치가 아닌 값을 가진 행들을 선택하고,
해당 행들의 'Cabin' 값을 공백을 기준으로 분할한 후,
분할된 값들을 가공하여 'Cabin_Type'과 'Cabin_No'로
나누는 과정을 나타냅니다.

1. `df_titanic.loc[df_titanic['Cabin'].notna(), 'Cabin'].str.split(' ')`:
이 부분은 'Cabin' 열에서 결측치가 아닌 값을 가진 행들을 선택하고,
해당 행들의 'Cabin' 값을 공백을 기준으로 분할합니다.
각 값은 공백을 기준으로 분할되어 리스트로 반환됩니다.

2. `.apply(lambda x: x[0])`:
이 부분은 분할된 리스트의 각 원소 중 첫 번째 원소를 선택하는
Lambda 함수를 적용합니다.
각 행의 'Cabin' 값 중 첫 번째 원소를 선택하여
새로운 Series를 생성합니다.

3. `.apply(lambda x: pd.Series([x[:1], x[1:]], index=['Cabin_Type', 'Cabin_No']))`:
이 부분은 각 행의 'Cabin' 값을 가공하여 'Cabin_Type'과
'Cabin_No'로 나누는 Lambda 함수를 적용합니다.
Lambda 함수는 'Cabin' 값을 받아서 첫 글자를 'Cabin_Type'으로,
나머지를 'Cabin_No'로 하는 Series를 반환합니다.
- `x[:1]`은 각 'Cabin' 값의 첫 번째 문자를 선택합니다.
  이는 'Cabin' 값을 'Cabin_Type'으로 할당합니다.
- `x[1:]`은 각 'Cabin' 값의 첫 번째 문자 이후의 문자열을 선택합니다.
  이는 'Cabin' 값을 'Cabin_No'로 할당합니다.
- 이렇게 생성된 Series들을 하나의 DataFrame으로 합쳐줍니다.
```

```

결과적으로 이 코드는 'Cabin' 열에서 결측치가 아닌 값을 가진 행들의
'Cabin' 값을 가공하여 'Cabin_Type'과 'Cabin_No'로 나누어
새로운 DataFrame을 생성합니다.
'''

# 1. Cabin에서 미결측치만 가져오고 공백으로 나눕니다.
# 2. 맨 처음 나오는 Cabin 만을 남깁니다.
# 3. Cabin_Type과 Cabin_No로 나눕니다
df_titanic.loc[df_titanic['Cabin'].notna(), 'Cabin'].str.split(' ')\
    .apply(lambda x: x[0])\
    .apply(lambda x: pd.Series([x[:1], x[1:]],
                                index=['Cabin_Type', 'Cabin_No']))

```

Out[11]:

	Cabin_Type	Cabin_No
PassengerId		
2	C	85
4	C	123
7	E	46
11	G	6
12	C	103
...	...	...
872	D	35
873	B	51
880	C	50
888	B	42
890	C	148

204 rows × 2 columns

In [12]:

```
'''
이 코드는 타이타닉 데이터셋의 'Cabin' 열을 기반으로
'Cabin_Type'과 'Cabin_No' 열을 새로 생성하는 것입니다.
여기서 'Cabin' 열은 객실 번호를 나타내며,
이를 공백을 기준으로 분리하여 'Cabin_Type'과 'Cabin_No' 열을 생성합니다.

먼저, 코드는 'Cabin' 열이 비어 있지 않은 행만 선택합니다.
이를 통해 객실 번호가 있는 행만 골라냅니다.

그런 다음, 선택된 행들의 'Cabin' 값을 공백을 기준으로 분리합니다.
이 과정에서 각 행의 'Cabin' 값은 리스트로 분리됩니다.

그 다음, 각 행의 'Cabin' 값 리스트의 첫 번째 요소를 선택하고,
이를 'Cabin_Type' 열로 만듭니다.
이때, 첫 번째 문자를 객실의 유형을 나타내는 부분으로 추출합니다.

마지막으로, 'Cabin' 값 리스트의 두 번째 요소를 선택하여
객실 번호를 나타내는 'Cabin_No' 열로 만듭니다.
여기서는 첫 번째 문자를 'Cabin_Type'으로,
나머지 문자열을 'Cabin_No'로 지정합니다.

따라서 이 코드는 'Cabin' 열을 활용하여
'Cabin_Type'과 'Cabin_No' 열을 추출하는 과정을 나타냅니다.
-----
위의 코드는 'Cabin' 열의 값을 가공하여 새로운 열인
'Cabin_Type'과 'Cabin_No'를 생성하고, 해당 값들을 채우는 과정을 나타냅니다.

1. `df_titanic[['Cabin_Type', 'Cabin_No']] = ...`:
   이 부분은 DataFrame에 'Cabin_Type'과 'Cabin_No'라는
   두 개의 새로운 열을 추가하고, 이 열들에 값을 할당합니다.
2. `df_titanic.loc[df_titanic['Cabin'].notna(), 'Cabin'].str.split(' ')`:
   이 부분은 'Cabin' 열에서 결측치가 아닌 값을 가진 행들을 선택하고,
   해당 행들의 'Cabin' 값을 공백을 기준으로 분할합니다.
   각 값은 공백을 기준으로 분할되어 리스트로 반환됩니다.
3. `.apply(lambda x: x[0])`:
   이 부분은 분할된 리스트의 각 원소 중 첫 번째 원소를 선택하는
   Lambda 함수를 적용합니다.
   각 행의 'Cabin' 값 중 첫 번째 원소를 선택하여
   'Cabin_Type' 열의 값을 생성합니다.
4. `.apply(lambda x: pd.Series([x[:1], x[1:]], index=['Cabin_Type', 'Cabin_No']))`:
```

- 이 부분은 각 행의 'Cabin' 값을 가공하여 'Cabin_Type'과 'Cabin_No' 열을 생성하는 Lambda 함수를 적용합니다.
- ``x[:1]``은 각 'Cabin' 값의 첫 번째 문자를 선택합니다.
이는 'Cabin' 값을 'Cabin_Type' 열의 값으로 할당합니다.
 - ``x[1:]``은 각 'Cabin' 값의 첫 번째 문자 이후의 문자열을 선택합니다.
이는 'Cabin' 값을 'Cabin_No' 열의 값으로 할당합니다.
 - 이렇게 생성된 Series들을 하나의 DataFrame으로 합쳐줍니다.

결과적으로 이 코드는 'Cabin' 열에서 결측치가 아닌 값을 가진 행들의 'Cabin' 값을 가공하여 'Cabin_Type'과 'Cabin_No'라는 두 개의 새로운 열을 생성하고, 해당 열들에 값을 할당합니다.

위 코드는 다음과 같은 순서로 동작합니다:

1. ``df_titanic['Cabin'].notna()`:`
'Cabin' 열에서 결측치가 아닌 행들을 선택합니다.
이를 통해 결측치가 아닌 행들의 불리언 인덱스를 얻습니다.
2. ``df_titanic.loc[]`:`
위에서 얻은 불리언 인덱스를 사용하여
해당 조건을 만족하는 행들을 선택합니다.
3. ``df_titanic.loc[df_titanic['Cabin'].notna(), 'Cabin']`:`
결측치가 아닌 'Cabin' 값들로 이루어진 Series를 선택합니다.
4. ``str.split(' ')`:`
선택된 'Cabin' 값들을 공백을 기준으로 분할합니다.
각 값은 공백을 기준으로 분할되어 리스트로 반환됩니다.
5. ``apply(lambda x: x[0])`:`
분할된 리스트의 각 원소 중 첫 번째 원소를 선택합니다.
이는 'Cabin_Type' 열의 값으로 사용됩니다.
6. ``apply(lambda x: pd.Series([x[:1], x[1:]], index=['Cabin_Type', 'Cabin_No']))`:`
분할된 리스트를 가지고 'Cabin_Type'과 'Cabin_No' 열을 만드는
Lambda 함수를 적용합니다. 이때, 각 리스트의 첫 번째 원소는
'Cabin_Type'에, 나머지는 'Cabin_No'에 할당됩니다.
7. ``df_titanic[['Cabin_Type', 'Cabin_No']] = ...`:`
새로 생성된 'Cabin_Type'과 'Cabin_No' 열을 DataFrame에 추가합니다.

결과적으로, 'Cabin' 열의 값들을 공백을 기준으로 분할하여
'Cabin_Type'과 'Cabin_No' 열로 나누고,
해당 열들을 DataFrame에 추가하는 과정이 이루어집니다.

```
# Cabin_Type과 Cabin_No를 df_titanic에 넣어 줍니다.
df_titanic[['Cabin_Type', 'Cabin_No']] = \
    df_titanic.loc[df_titanic['Cabin'].notna(), 'Cabin'].str.split(' ')\
    .apply(lambda x: x[0])\
    .apply(lambda x: pd.Series([x[:1], x[1:]],
                               index=['Cabin_Type', 'Cabin_No']))
```

```
In [13]: # 최빈값 구하기: pd.Series.mode
df_titanic['Cabin_Type'].mode()
```

```
Out[13]: 0    C
dtype: object
```

pd.Series.mode() : 최빈값을 구합니다.

복수의 최빈값이 나오는 경우를 고려하고 있어 Series로 반환됩니다.

```
In [14]: # 최빈값은 복수일 수 있으므로, Series를 반환합니다.
# 요건에 맞게 사전 순 정렬을 합니다.
# 그 다음에 첫번째 요소를 가져 옵니다.

df_titanic['Cabin_Type'].mode().sort_values().iloc[0]
```

```
Out[14]: 'C'
```

```
In [15]: '''
이 코드는 타이타닉 데이터셋을 'Pclass' (객실 등급)로 그룹화한 후,
각 그룹 내에서 'Cabin_Type' 열의 가장 빈번하게
나타나는 값을 찾는 작업을 수행합니다.
먼저, `groupby('Pclass')`를 사용하여 데이터를 'Pclass' 열을 기준으로 그룹화합니다.
이렇게 하면 데이터가 각각의 객실 등급에 따라 분리됩니다.
그런 다음, 'Cabin_Type' 열을 각 그룹에 대해 분석합니다.
여기서 `agg()` 함수를 사용하여 그룹별로 특정한 집계 함수를 적용합니다.
이 경우에는 람다 함수가 사용되었는데,
이 람다 함수는 각 그룹의 'Cabin_Type' 값 중에서 최빈값(mode)을 찾습니다.
`mode()` 함수는 최빈값을 찾아주며, 여기에 `sort_values()` 함수를 적용하여
값들을 정렬한 후, `iloc[0]`를 사용하여 정렬된 값들 중에서 첫 번째 값을 선택합니다.
이것은 최빈값이 여러 개일 경우에도 가장 작은 값을 선택하기 위함입니다.
따라서 이 코드는 각 객실 등급별로 가장 빈번하게 등장하는
'Cabin_Type' 값을 찾는 작업을 수행합니다.
'''
```

```
'''
# 이 루틴을 Pclass 별로 수행합니다.

df_titanic.groupby('Pclass')['Cabin_Type'].agg(
    lambda x: x.mode().sort_values().iloc[0]
)
```

```
Out[15]: Pclass
1      C
2      F
3      F
Name: Cabin_Type, dtype: object
```

```
In [16]: '''
코드는 타이타닉 데이터셋을 'Pclass' (객실 등급)로 그룹화한 후,
각 그룹 내에서 'Cabin_Type' 열의 결측값을 해당 그룹의
최빈값(mode)으로 대체하는 작업을 수행합니다.

먼저, `groupby('Pclass')`를 사용하여 데이터를 'Pclass' 열을
기준으로 그룹화합니다. 이렇게 하면 데이터가 각각의 객실 등급에 따라 분리됩니다.

그런 다음, 'Cabin_Type' 열을 각 그룹에 대해 분석합니다.
여기서 `transform()` 함수를 사용하여 각 그룹 내에서 값을 변환합니다.
이 변환은 람다 함수를 통해 이루어지는데,
람다 함수는 각 그룹의 'Cabin_Type' 값을 받아 결측값을 최빈값으로 채웁니다.

`mode()` 함수는 최빈값을 찾아주며, 여기에 `sort_values()` 함수를 적용하여
값들을 정렬한 후, `iloc[0]`를 사용하여 정렬된 값들 중에서
첫 번째 값을 선택합니다. 이것은 최빈값이 여러 개일 경우에도
가장 작은 값을 선택하기 위함입니다.

따라서 이 코드는 각 객실 등급별로 'Cabin_Type' 열의 결측값을
해당 그룹의 최빈값으로 대체하는 작업을 수행합니다.
이렇게 함으로써 데이터셋 내에서
객실 등급에 따른 일관성 있는 대체가 이루어집니다.
'''

# 이를 이용해 결측 처리한 Series를 만듭니다.

df_titanic.groupby('Pclass')['Cabin_Type'].transform(
```

```
lambda x: x.fillna(x.mode().sort_values().iloc[0])
)
```

```
Out[16]: PassengerId
1      F
2      C
3      F
4      C
5      F
..
887    F
888    B
889    F
890    C
891    F
Name: Cabin_Type, Length: 891, dtype: object
```

```
In [17]: '''
위 코드는 타이타닉 데이터셋에서 'Cabin_Type' 열의 결측값을
각 행이 속한 객실 등급('Pclass')에 따른
최빈값(mode)으로 대체하는 작업을 수행합니다.
먼저, `groupby('Pclass')`를 사용하여 데이터를
'Pclass' 열을 기준으로 그룹화합니다.
그 후, 각 그룹 내에서 'Cabin_Type' 열의
최빈값을 찾는 작업이 이루어집니다.
이를 위해 람다 함수가 사용되었는데, 람다 함수는
각 그룹의 'Cabin_Type' 값을 받아 최빈값을 찾습니다.
여기서 `mode()` 함수는 최빈값을 찾아주며,
`sort_values()` 함수를 적용하여 값들을 정렬한 후,
`iloc[0]`를 사용하여 정렬된 값들 중에서 첫 번째 값을 선택합니다.
이것은 최빈값이 여러 개일 경우에도 가장 작은 값을 선택하기 위함입니다.
그런 다음, `fillna()` 함수를 사용하여 'Cabin_Type' 열의 결측값을
위에서 찾은 각 그룹별 최빈값으로 대체합니다.
이때, `transform()` 함수를 사용하여 각 행이 속한 그룹의
최빈값으로 대체합니다. 따라서 결측값이 속한 행의
'Pclass'에 해당하는 그룹의 최빈값으로 대체됩니다.
이를 통해 'Cabin_Type' 열의 결측값을 일관되게 대체하여
데이터의 완결성을 보장할 수 있습니다.
'''

# fillna에는 Series를 입력으로 받을 수 있습니다.
```

```
# fillna에 Series를 전달하면,
# 결측인 요소에 해당하는 요소를 Series에서 가져와서 대체합니다.
df_titanic['Cabin_Type'].fillna(
    df_titanic.groupby('Pclass')['Cabin_Type'].transform(
        lambda x: x.mode().sort_values().iloc[0]
    )
)
```

```
Out[17]: PassengerId
1      F
2      C
3      F
4      C
5      F
..
887    F
888    B
889    F
890    C
891    F
Name: Cabin_Type, Length: 891, dtype: object
```

```
In [18]: '''
코드는 타이타닉 데이터셋에서 'Cabin_Type' 열의 결측값을
각 행이 속한 객실 등급('Pclass')에 따른
최빈값(mode)으로 대체하는 작업을 수행합니다.
먼저, `groupby('Pclass')`를 사용하여 데이터를 'Pclass' 열을
기준으로 그룹화합니다. 그 후, 각 그룹 내에서 'Cabin_Type' 열의
최빈값을 찾는 작업이 이루어집니다.
이를 위해 람다 함수가 사용되었는데,
람다 함수는 각 그룹의 'Cabin_Type' 값을 받아 최빈값을 찾습니다.
여기서 `mode()` 함수는 최빈값을 찾아주며,
`sort_values()` 함수를 적용하여 값들을 정렬한 후,
`iloc[0]`를 사용하여 정렬된 값들 중에서 첫 번째 값을 선택합니다.
이것은 최빈값이 여러 개일 경우에도 가장 작은 값을 선택하기 위함입니다.
그런 다음, `transform()` 함수를 사용하여 각 행이 속한 그룹의
결측값을 해당 그룹의 최빈값으로 대체합니다.
이를 통해 'Cabin_Type' 열의 결측값을 각 행이 속한
객실 등급에 해당하는 그룹의 최빈값으로 일관되게 대체할 수 있습니다.
마지막으로, 이렇게 결측값이 대체된 값을 'Cabin_Type' 열에 할당하여
해당 열을 업데이트합니다.
'''
```

```

따라서 이 코드는 'Cabin_Type' 열의 결측값을 각 행이 속한
객실 등급에 해당하는 그룹의 최빈값으로 일관되게 대체하여
데이터의 완결성을 보장하는 역할을 합니다.
'''

# 결측 처리 결과를 반영합니다.

df_titanic['Cabin_Type'] =
    df_titanic.groupby('Pclass')['Cabin_Type'].transform(
        lambda x: x.fillna(x.mode().sort_values().iloc[0])
    )

```

```
In [19]: df_titanic['Cabin_Type'].value_counts()
```

```

Out[19]: F    660
         C     99
         B     47
         D     33
         E     32
         A     15
         G      4
         T      1
         Name: Cabin_Type, dtype: int64

```

수치형 데이터

1. 평균(중심 통계량: 평균, 중앙값, 최빈값)으로 대체

```

In [20]: # 평균으로 대체하여 반영합니다.
df_titanic['Age'] = df_titanic['Age'].fillna(df_titanic['Age'].mean())

```

2. 보간법(Interpolation)

순서가 있는 데이터(시간순, 위치순)인 경우에 앞, 뒤 값을 활용하여 결측치를 대체합니다.

`pd.Series.interpolate`, `pd.DataFrame.interpolate`

In [11]: `Image('9.png')`

Out[11]: **Linear Interpolation**

$$\frac{y-y_0}{x-x_0} = \frac{y_1-y_0}{x_1-x_0}$$

$$y = y_0 + (x - x_0) \frac{y_1 - y_0}{x_1 - x_0}$$

```
In [ ]: # 샘플 데이터를 만듭니다.
s = pd.Series([1, np.nan, np.nan, 7], index=[1, 2.5, 3, 4])

fig, axes = plt.subplots(1, 2, figsize=(12, 4))
# 결측이 아닌 두점을 잇는 선을 그립니다.
# linear: x 값은 고려하지 않고, 등간격이라는 가정하에 선형 보간을 합니다.
# index: index의 값이 x값이라고 가정하고 선형 보간을 합니다.
for i, ax in zip(['linear', 'index'], axes):
    ax.plot(s.loc[s.notna()])

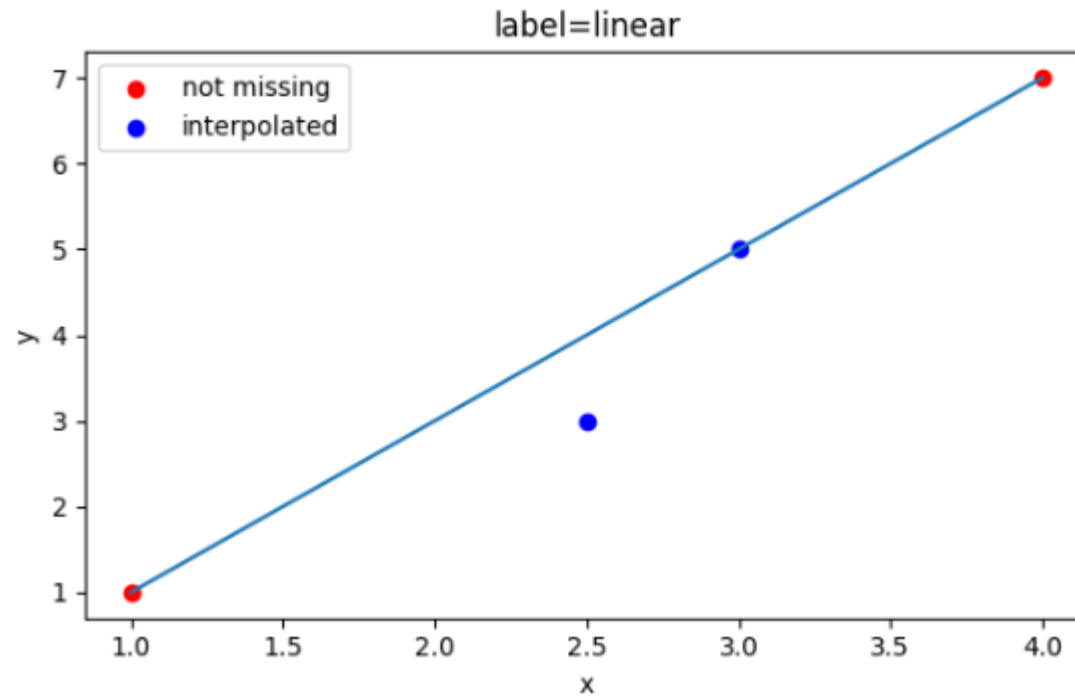
    # 결측이 아닌 값을 찍습니다.
    s.loc[s.notna()].pipe(
        lambda x: ax.scatter(x=x.index, y=x.values, color='red',
                              label='not missing')
    )
    # 결측인 값을 보간한 값을 찍습니다.
    s.interpolate(method=i).loc[s.isna()].pipe(
        lambda x: ax.scatter(x=x.index, y=x.values, color='blue',
                              label='interpolated')
    )

ax.legend()
```

```
ax.set_xlabel('x')  
ax.set_ylabel('y')  
ax.set_title("label=" + i)  
plt.tight_layout()  
plt.show()
```

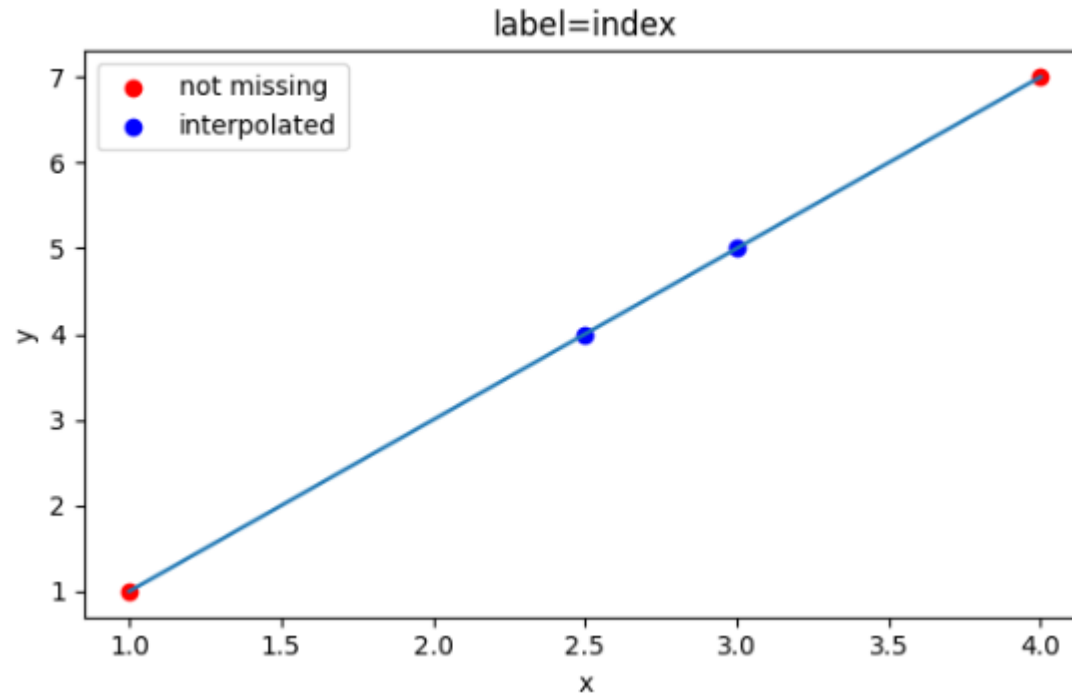
In [12]: Image('10.png')

Out[12]:



In [13]: Image('11.png')

Out[13]:



Regression

- 다른 변수를 이용하여 결측치를 예측하는 모델을 만들어,
모델의 예측값으로 결측치를 대체합니다.

[Ex.3]

Age'의 결측값을 'Pclass', 'Sex', 'SibSp', 'Parch', 'Fare', 'Embarked' 를
이용하여 예측 모델을 만들어 예측치로 결측값을 대체합니다.
예측 모델은 Random Forest Regression 모델로 하고,
Pclass, Sex, Embarked는 가변수화 합니다.

나머지 입력값은 그대로 모델에 사용합니다.

```
In [22]: # Age로 예시를 듭니다.
# Age를 위에서 결측을 처리했으므로,
# Age를 다시 결측 처리하기 위해 원래 데이터를 불러옵니다.
df_titanic = pd.read_csv('data/titanic.csv')

# Embarked는 최빈값으로 결측을 처리합니다.
df_titanic['Embarked']
    = df_titanic['Embarked'].fillna(df_titanic['Embarked'].mode().iloc[0])
```

```
In [ ]: '''
`make_pipeline()` 함수는 여러 개의 변환기(Transformer)와
추정기(Estimator)를 순차적으로 연결하여 하나의 파이프라인을
만들어주는 함수입니다.
이를 통해 데이터 변환 및 모델 학습 과정을 간편하게 처리할 수 있습니다.
```

특징:

1. 간편한 파이프라인 생성:
파이프라인을 만들 때 별도로 이름을 부여하거나
복잡한 작업 없이 간단하게 파이프라인을 생성할 수 있습니다.
2. 순차적으로 변환 및 학습:
파이프라인 내의 각 변환기와 추정기는
순차적으로 연결되어 데이터가 변환되고 모델이 학습됩니다.
3. 유연성:
다양한 변환기와 추정기를 조합하여
파이프라인을 구성할 수 있습니다.

인자:

```
`make_pipeline()` 함수는 변환기(Transformer)와
추정기(Estimator)를 인자로 받습니다.
각각의 변환기와 추정기는 파이프라인에서 순차적으로 연결됩니다.
```

리턴값:

```
`make_pipeline()` 함수는 변환기와 추정기가
순차적으로 연결된 파이프라인을 반환합니다.
```

사용법:

```

from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

# 변환기와 추정기를 순차적으로 연결하여 파이프라인 생성
pipeline = make_pipeline(
    StandardScaler(), # 변환기 1
    LogisticRegression() # 추정기
)

# 생성한 파이프라인을 사용하여 데이터 변환 및 모델 학습
pipeline.fit(X_train, y_train)

위 코드에서는 `StandardScaler()`를 사용하여 데이터를 표준화하고,
`LogisticRegression()`을 사용하여 로지스틱 회귀 모델을 학습하는
파이프라인을 생성하였습니다.
`fit()` 메서드를 사용하여 파이프라인을
훈련 데이터에 적합시킬 수 있습니다.

파이프라인을 사용하면 데이터 전처리 및 모델 학습과정을
간편하게 처리할 수 있으며, 이를 통해 코드의
가독성과 유지 보수성을 향상시킬 수 있습니다.
'''

# 전처리 단계의 출력을 바로 모델에 전달 될 수 있도록
# make_pipeline을 사용하여 Pipeline을 구성토록 합니다.
from sklearn.pipeline import make_pipeline

```

```

In [ ]: '''
`ColumnTransformer()` 함수는 서로 다른 열에 대해
서로 다른 변환을 적용할 수 있도록 하는 기능을 제공합니다.
이를 통해 데이터셋 내의 여러 열에 대해
각기 다른 전처리 작업을 수행할 수 있습니다.

특징:
1. 다중 열 변환: 데이터셋 내의 여러 열에 대해
   각기 다른 전처리 작업을 적용할 수 있습니다.
2. Pipeline과 호환: 각 열에 대한 전처리 과정을
   파이프라인으로 정의하고 적용할 수 있습니다.
3. 이름 지정: 각 변환기에 대해 명시적으로 이름을 부여하여

```

코드 가독성을 높일 수 있습니다.

인자:

`ColumnTransformer()` 함수는 변환기(Transformer)와 열 매핑(Feature mapping)을 지정하는 `transformers` 인자를 필요로 합니다. `transformers` 인자는 리스트 형태로 각 변환기와 열 매핑을 포함하고 있습니다.

리턴값:

`ColumnTransformer()` 함수는 변환된 데이터셋을 반환합니다.

사용법:

```
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.pipeline import Pipeline
```

변환할 열과 적용할 변환기 정의

```
column_transformer = ColumnTransformer(
    transformers=[
        # 숫자형 열에 대한 표준화
        ('numeric', StandardScaler(),
         ['numerical_column_1', 'numerical_column_2']),
        # 범주형 열에 대한 원-핫 인코딩
        ('categorical', OneHotEncoder(), ['categorical_column'])
    ],
    remainder='passthrough' # 변환하지 않을 열은 그대로 유지
)
```

데이터 변환

```
transformed_data = column_transformer.fit_transform(X)
```

위 코드에서는 `ColumnTransformer()`를 사용하여 데이터셋의 'numerical_column_1', 'numerical_column_2' 열에 대해 표준화(StandardScaler)를 적용하고, 'categorical_column' 열에 대해 원-핫 인코딩(OneHotEncoder)을 적용하는 변환기를 정의하였습니다. `remainder='passthrough'` 인자는 변환하지 않을 열은 그대로 유지하도록 지정하였습니다.

이렇게 정의한 `ColumnTransformer`를 데이터에 적용하여 각 열에 대해 정의한 변환기를 순차적으로 적용합니다. 이를 통해 데이터셋 내의 다양한 유형의 열에 대해

각기 다른 전처리 과정을 쉽게 적용할 수 있습니다.
'''

```
# ColumnTransformer를 사용하여
# 변수별 전처리 과정을 설정할 수 있도록 합니다.
from sklearn.compose import ColumnTransformer
```

```
In [ ]: # 가변수 처리기를 가져옵니다.
from sklearn.preprocessing import OneHotEncoder

from sklearn.ensemble import RandomForestRegressor

'''
```

코드는 다음과 같은 작업을 수행하는
ColumnTransformer 객체를 생성합니다.

1. 'Pclass', 'Sex', 'Embarked' 열에 대해
원-핫 인코딩(OneHotEncoder)을 적용합니다.
2. 'SibSp', 'Parch', 'Fare' 열에 대해서는
변환을 수행하지 않고 그대로 유지합니다.

이를 수행하기 위해 ColumnTransformer() 함수를 사용합니다.
여기서 변환기(Transformer)와
열 매핑(Feature mapping)은 리스트 형태로 전달됩니다.

1. `('ohe', OneHotEncoder(), ['Pclass', 'Sex', 'Embarked'])`:
첫 번째 튜플에서는 원-핫 인코딩(OneHotEncoder)을 적용할 열과
그 처리기를 지정합니다. 'Pclass', 'Sex', 'Embarked' 열에 대해
각각 OneHotEncoder를 적용합니다.
'ohe'는 이 변환기에 대한 이름으로 사용됩니다.
2. `('pt', 'passthrough', ['SibSp', 'Parch', 'Fare'])`:
두 번째 튜플에서는 변환을 수행하지 않고 열을 그대로 유지(passthrough)할
열과 처리기를 지정합니다. 'SibSp', 'Parch', 'Fare' 열에 대해서는
어떠한 변환도 수행하지 않습니다.
'pt'는 이 열에 대한 이름으로 사용됩니다.

따라서 이 ColumnTransformer 객체는 주어진 데이터셋에 대해
'Pclass', 'Sex', 'Embarked' 열에 대해 원-핫 인코딩을 적용하고,
'SibSp', 'Parch', 'Fare' 열에 대해서는
그대로 유지하는 변환 작업을 수행합니다.

```
'''
# OneHotEncoder는 Pclass, Sex, Embarked
# SibSp, Parch, Fare는 처리 없이 통과 시킵니다.
ct = ColumnTransformer([
    # (이름, 처리기, 처리 대상 변수)
    ('ohe', OneHotEncoder(), ['Pclass', 'Sex', 'Embarked']),
    ('pt', 'passthrough', ['SibSp', 'Parch', 'Fare'])
])
```

In []:

```
'''
코드는 머신러닝 모델링 과정에서 데이터 전처리와
모델 학습을 통합하여 수행하기 위해
파이프라인을 구축하는 작업을 수행합니다.

1. ColumnTransformer 설정:
- 앞서 설명한대로, `ColumnTransformer` 객체 `ct`는
  다양한 열에 대해 다른 전처리 작업을 수행하기 위해 사용됩니다.
  `Pclass`, `Sex`, `Embarked` 열에 대해서는 원-핫 인코딩을 적용하고,
  `SibSp`, `Parch`, `Fare` 열에 대해서는 그대로 유지하는 역할을 합니다.

2. RandomForestRegressor 설정:
- RandomForestRegressor는 랜덤 포레스트 모델을
  구축하는 추정기(Estimator)입니다.
- `n_estimators=100`은 랜덤 포레스트에서 사용할
  의사 결정 나무의 개수를 100개로 설정합니다.
- `random_state=123`은 모델을 학습할 때 사용되는
  난수 시드(seed)를 설정하여 실행할 때마다
  동일한 결과를 얻을 수 있도록 합니다.

3. 파이프라인 구축:
- `make_pipeline()` 함수를 사용하여 앞서 정의한 ColumnTransformer 객체와
  RandomForestRegressor 모델을 순차적으로 연결하여
  파이프라인을 생성합니다.
- 이렇게 생성된 파이프라인 `reg_rf`는 먼저 데이터에
  ColumnTransformer를 적용한 후에
  RandomForestRegressor 모델을 학습시킵니다.
- 파이프라인을 사용함으로써 데이터의 전처리와 모델 학습을
  간편하게 처리할 수 있으며,
  코드의 가독성과 유지보수성을 높일 수 있습니다.
```

따라서 위 코드는 데이터의 전처리와 모델 학습 과정을 하나의 파이프라인으로 통합하여 효율적으로 관리하는데 사용됩니다.

```
reg_rf = make_pipeline(
    ct,
    RandomForestRegressor(
        # RandomForest에서 사용할 의사 결정 나무의 수를 100개로 합니다.
        n_estimators=100,
        # random_state에 값을 지정하여 실행시 동일한 결과를 얻게 설정합니다.
        random_state=123)
)
```

In []:

코드는 다음과 같은 작업을 수행합니다:

1. 입력변수 리스트 정의:

- `X\_rf` 변수에는 모델의 입력변수로 사용될 열들의 리스트가 정의되어 있습니다.
여기서는 'Pclass', 'Sex', 'Embarked', 'SibSp', 'Parch', 'Fare' 열을 사용합니다.

2. 파이프라인 구축 및 모델 학습:

- `df\_titanic.loc[df\_titanic['Age'].notna()]`를 통해 'Age' 열에 결측값이 없는 데이터만 선택합니다.
이를 통해 모델의 학습을 위한 데이터셋을 준비합니다.
- `pipe()` 함수를 사용하여 데이터프레임을 함수 또는 메서드에 전달합니다.
- 람다 함수를 사용하여 데이터프레임에 대해 다음 작업을 수행합니다:
 - `x[X\_rf]`: 입력변수 리스트에 해당하는 열들을 선택하여 모델의 입력 데이터로 사용합니다.
 - `x['Age']`: 타겟 변수로 사용할 'Age' 열을 선택합니다.
 - `reg\_rf.fit(x[X\_rf], x['Age'])`: 앞서 구축한 RandomForestRegressor 모델 `reg\_rf`를 선택한 입력변수와 타겟변수에 대해 학습시킵니다.
이때, 입력변수로는 선택한 열들을, 타겟변수로는 'Age' 열을 사용합니다.

따라서 아래 코드는 'Age' 열에 결측값이 없는 데이터를 사용하여 RandomForestRegressor 모델을 학습시키는 작업을 수행합니다.

```
'''
# 입력변수 리스트 입니다.
X_rf = ['Pclass', 'Sex', 'Embarked', 'SibSp', 'Parch', 'Fare']
# Regression 도식에 중심을 두겠습니다.
# 모델에 대한 검증은 생략하고 바로 학습을 합니다.

# 모델을 학습시킵니다.
df_titanic.loc[df_titanic['Age'].notna()].pipe(
    lambda x: reg_rf.fit(x[X_rf], x['Age'])
)

'''

위 루틴과 동일한 루틴입니다.
위 구문은 아래 구문에 df_titanic['Age'].notna()로
필터링 2회 했던 것을 pipe를 통해 줄여낸 것입니다.
reg_rf.fit(
    df_titanic.loc[df_titanic['Age'].notna()], X_rf,
    df_titanic.loc[df_titanic['Age'].notna()], 'Age'
)
'''
```

In [23]:

```
'''
코드는 'Age' 열의 결측값을 RandomForestRegressor 모델을
사용하여 예측한 값으로 대체하는 작업을 수행합니다.

1. `df_titanic.loc[df_titanic['Age'].isna(), 'Age']`:
   'Age' 열의 결측값을 선택합니다.
2. `df_titanic.loc[df_titanic['Age'].isna(), X_rf]`:
   결측값이 있는 행들에 대해 입력변수 리스트에 해당하는
   열들을 선택하여 예측에 사용할 데이터를 구성합니다.
3. `reg_rf.predict(...)` :
   RandomForestRegressor 모델을 사용하여 위에서 구성한
   데이터를 바탕으로 'Age' 열의 결측값을 예측합니다.
4. 이 예측 결과를 결측값이 있는 행의 'Age' 열에
   대입하여 결측값을 대체합니다.

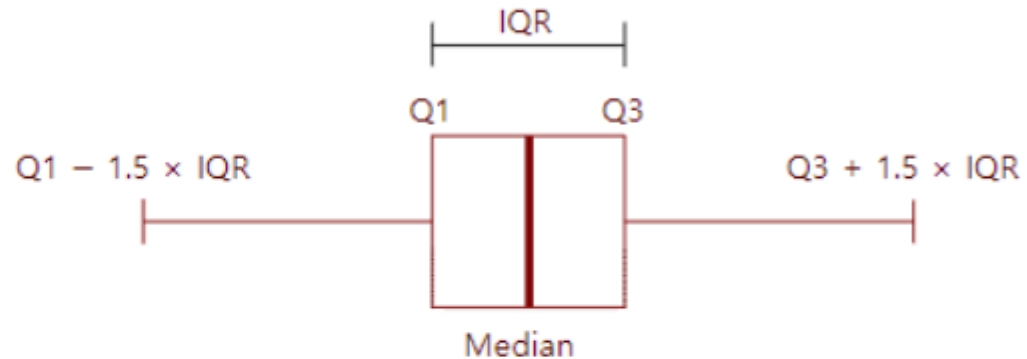
따라서 아래 코드는 RandomForestRegressor 모델을 사용하여
결측값을 예측하고, 그 예측 결과로 결측값을 대체하는 작업을 수행합니다.
이를 통해 데이터셋 내의 'Age' 열에 있는 결측값을 보완할 수 있습니다.
'''
```

```
# 결측값에 예측 결과를 적용합니다.
df_titanic.loc[df_titanic['Age'].isna(), 'Age'] = \
    reg_rf.predict(df_titanic.loc[df_titanic['Age'].isna(), X_rf])
```

In [14]: Image('12.png')

Out[14]: **이상치 처리**

- **IQR**(Inter Quantile Range)을 통한 이상치 판별



$$Q_1 - 1.5 \times IQR \leq X \leq Q_3 + 1.5 \times IQR$$

Q_1 : 제1사분위수, Q_3 : 제3사분위수

$$IQR = Q_3 - Q_1$$

IQR에 곱해진 1.5는 경우에 따라 달라질 수 있습니다.

이상치 제거

- 이상치로 판단된 데이터를 제거합니다.

In [15]: `Image('13.png')`

Out[15]: **[Ex.4]**

df_berry 셋의 fruitset, fruitmass, seeds 가

각 변수별로 Q_1 , Q_3 , IQR 을 구하여, 이상치 여부를 판별합니다.

$$Q_1 - 1.5 \times IQR \leq X \leq Q_3 + 1.5 \times IQR$$

이중에 하나라도 이상치라면 제거하여 df_berry2셋을 만듭니다.

```
In [3]: '''
이 코드는 주어진 데이터프레임 `df_berry`에서 세 가지 변수
(`fruitset`, `fruitmass`, `seeds`)에 대한 이상치를 탐지하기 위한 작업을 수행합니다.
이를 위해 주어진 변수의 사분위수 범위(Interquartile Range, IQR)를 계산하고,
이를 이용하여 이상치의 상한선(Upper Bound, UB)과 하한선(Lower Bound, LB)을 결정합니다.

여기서 각 단계를 자세히 설명하겠습니다:
1. `df_berry[['fruitset', 'fruitmass', 'seeds']]`:
   주어진 데이터프레임에서 `fruitset`, `fruitmass`, `seeds` 열만 선택합니다.
2. `.apply(lambda x: x.quantile([0.25, 0.75]).rename({0.25: 'Q1', 0.75: 'Q3'}))`:
   각 변수(열)에 대해 1사분위수(Q1)와 3사분위수(Q3)를 계산합니다.
   이는 데이터의 하위 25%와 상위 25%에 해당하는 값들을 의미합니다.
   `.rename({0.25: 'Q1', 0.75: 'Q3'})`는 결과를 이해하기 쉽도록
   컬럼 이름을 변경합니다.
3. `.T`: 계산된 사분위수를 행과 열을 바꾸어 줍니다.
4. `.assign(IQR = lambda x: x['Q3'] - x['Q1'])`:
   사분위 범위(IQR)를 계산합니다.
   IQR은 Q3에서 Q1을 뺀 값으로, 데이터의 중간 50% 범위를 나타냅니다.
5. `.assign(LB = lambda x: x['Q1'] - 1.5 * x['IQR'])`:
   하한선(Lower Bound, LB)을 계산합니다. 일반적으로 하한선은 Q1에서 1.5배의
   IQR을 뺀 값으로 정의됩니다.
   이 범위를 벗어나는 값들은 이상치로 간주될 수 있습니다.
```

```
6. `.assign(UB = lambda x: x['Q3'] + 1.5 * x['IQR'])`:  
   상한선(Upper Bound, UB)을 계산합니다. 마찬가지로, 일반적으로 상한선은  
   Q3에 1.5배의 IQR을 더한 값으로 정의됩니다.  
   이 범위를 벗어나는 값들은 이상치로 간주될 수 있습니다.
```

이러한 절차를 통해 각 변수에 대한 이상치의 상한선과 하한선이 계산되며,
나중에 이를 사용하여 이상치를 식별할 수 있습니다.

```
df_iqr = df_berry[['fruitset', 'fruitmass', 'seeds']].apply(  
    # 각 변수의 1사분위와 3사분위를 구합니다.  
    lambda x: x.quantile([0.25, 0.75]).rename({0.25: 'Q1', 0.75: 'Q3'})  
)  
.T.  
.assign(  
    IQR = lambda x: x['Q3'] - x['Q1'] # IQR을 구합니다.  
)  
.assign(  
    LB = lambda x: x['Q1'] - 1.5 * x['IQR'], # 하한선을 계산합니다.  
    UB = lambda x: x['Q3'] + 1.5 * x['IQR'] # 상한선을 계산합니다.  
)  
df_iqr
```

```
Out[3]:
```

	Q1	Q3	IQR	LB	UB
fruitset	0.454725	0.561297	0.106571	0.294868	0.721153
fruitmass	0.416281	0.476149	0.059869	0.326478	0.565952
seeds	33.116091	39.239668	6.123577	23.930726	48.425033

```
In [4]: # 각변수가 하한선 또는 상한선에서 벗어났는지 확인합니다.  
df_is_outlier = df_berry[['fruitset', 'fruitmass', 'seeds']].apply(  
    lambda x: (x < df_iqr.loc[x.name, 'LB']) |  
              (x > df_iqr.loc[x.name, 'UB'])  
)  
df_is_outlier
```

Out[4]:

	fruitset	fruitmass	seeds
Row#			
0	False	False	False
1	False	False	False
2	False	False	False
3	False	False	False
4	False	False	False
...	...	...	...
772	False	False	False
773	False	False	False
774	False	False	False
775	False	False	False
776	False	False	False

777 rows × 3 columns

```
In [5]: '''
이 코드는 이전에 계산한 이상치의 여부를 확인하여
이상치가 있는 행을 선택합니다.
선택된 행에서는 `df_berry` 데이터프레임의
`fruitset`, `fruitmass`, `seeds` 열만을 가져옵니다.

1. `df_is_outlier.any(axis=1)`:
   `df_is_outlier`는 이전에 계산한 이상치 여부에 대한
   데이터프레임입니다.
   `any(axis=1)`는 각 행에 대해 어떤 열(변수)이라도
   이상치를 가지고 있는지를 확인합니다.
   따라서 이 코드는 각 행에 대해 이상치가 하나 이상 있는지를 확인하고,
   그 결과를 불리언(boolean) 형태로 반환합니다.
2. `df_berry.loc[...]`:
```

이상치가 있는 행을 선택하기 위해 ``loc``를 사용합니다.
이전 단계에서 반환된 불리언 값에 따라 행이 선택됩니다.
즉, 이상치가 있는 행만 선택됩니다.

3. ``[...]``:

선택된 행에서는 ``['fruitset', 'fruitmass', 'seeds']``
열만을 선택하여 결과를 반환합니다.

결론적으로, 이 코드는 이전에 계산한 이상치 여부를 확인하고,
이상치가 있는 행에서 ``fruitset``, ``fruitmass``, ``seeds`` 열의
데이터를 선택하여 반환합니다.

'''

이상치가 하나라도 있는 행을 뽑아 봅니다.

`df_berry.loc[df_is_outlier.any(axis=1), ['fruitset', 'fruitmass', 'seeds']]`

Out[5]:

	fruitset	fruitmass	seeds
Row#			
6	0.284443	0.352186	26.101179
7	0.246568	0.342826	25.042361
16	0.288159	0.359459	26.487322
17	0.279536	0.354415	26.282356
18	0.233554	0.335339	24.320627
32	0.262140	0.349354	25.433530
47	0.283055	0.355876	26.054692
766	0.192732	0.311921	22.079199
768	0.249335	0.320727	23.412776

In [6]: '''

이 코드는 이전에 계산한 이상치의 여부를 확인하여
이상치가 있는 행을 데이터프레임 ``df_berry``에서 삭제하는 작업을 수행합니다.
이를 통해 이상치가 있는 행이 제거된 새로운 데이터프레임 ``df_berry2``가 생성됩니다.

1. `df_is_outlier.any(axis=1)`:
이전에 계산한 이상치 여부에 대한 데이터프레임 `df_is_outlier`를 사용합니다.
`any(axis=1)`는 각 행에 대해 어떤 열(변수)이라도
이상치를 가지고 있는지를 확인합니다.
따라서 이 코드는 각 행에 대해 이상치가 하나 이상 있는지를 확인하고,
그 결과를 불리언(boolean) 형태로 반환합니다.
2. `df_berry.loc[...]`: 이상치가 있는 행을 선택하기 위해 `loc`를 사용합니다.
이전 단계에서 반환된 불리언 값에 따라 이상치가 있는 행이 선택됩니다.
3. `.index`:
선택된 행의 인덱스를 가져옵니다.
4. `df_berry.drop(index=...)`:
`drop` 함수를 사용하여 이상치가 있는 행을 삭제합니다.
`index` 매개변수에 이상치가 있는 행의 인덱스를 전달하여 해당 행들을 삭제합니다.

결론적으로, 이 코드는 이전에 계산한 이상치 여부를 확인하고,
이상치가 있는 행을 데이터프레임에서 삭제하여
새로운 데이터프레임 `df_berry2`를 생성합니다.
...

*# 이상치에 해당하는 인덱스로,
이상치를 제거한 DataFrame을 df_berry2로 지정합니다.*

```
df_berry2 = df_berry.drop(index=df_berry.loc[df_is_outlier.any(axis=1)].index)
df_berry2
```

Out[6]:

	clonesize	honeybee	bumbles	andrena	osmia	MaxOfUpperTRange	MinOfUpperTRange	AverageOfUpperTRange	MaxOfLowerTF
Row#									
0	37.5	0.750	0.250	0.250	0.250	86.0	52.0	71.9	
1	37.5	0.750	0.250	0.250	0.250	86.0	52.0	71.9	
2	37.5	0.750	0.250	0.250	0.250	94.6	57.2	79.0	
3	37.5	0.750	0.250	0.250	0.250	94.6	57.2	79.0	
4	37.5	0.750	0.250	0.250	0.250	86.0	52.0	71.9	
...	...	...	...	...	...	...	...	...	...
772	10.0	0.537	0.117	0.409	0.058	86.0	52.0	71.9	
773	40.0	0.537	0.117	0.409	0.058	86.0	52.0	71.9	
774	20.0	0.537	0.117	0.409	0.058	86.0	52.0	71.9	
775	20.0	0.537	0.117	0.409	0.058	89.0	39.0	65.6	
776	20.0	0.537	0.117	0.409	0.058	89.0	39.0	65.6	

768 rows × 17 columns



범위 한정(clipping)

하한값을 벗어난 경우 하한값으로 대체하고

상한값을 벗어나 경우 상한값으로 대체시킵니다.

pd.Series.clip

In [7]: `Image('14.png')`

Out[7]: **[Ex.5]**

df_berry 셋의 fruitset, fruitmass, seeds 가

각 변수별로 Q_1 , Q_3 , IQR 을 구하여, 이상치 여부를 판별을 합니다.

$$Q_1 - 1.5 \times IQR \leq X \leq Q_3 + 1.5 \times IQR$$

하한값을 벗어났을 경우 하한값으로 대체하고,

상한값을 벗어났을 경우는 상한값으로 대체하여 df_berry3셋을 만듭니다.

```
In [28]: # Ex.4에서 사용했던 df_iqr을 사용해봅니다.
df_iqr
```

```
Out[28]:
```

	Q1	Q3	IQR	LB	UB
fruitset	0.454725	0.561297	0.106571	0.294868	0.721153
fruitmass	0.416281	0.476149	0.059869	0.326478	0.565952
seeds	33.116091	39.239668	6.123577	23.930726	48.425033

```
In [29]: '''
이 코드는 이상치를 제거한 데이터프레임 `df_berry3`의
`fruitset`, `fruitmass`, `seeds` 열에 대해 적용되며,
각 열의 값들을 이전에 계산한 이상치의 상한선(UB)과 하한선(LB) 사이의
값으로 잘라(clip) 변경합니다.

1. `df_berry3[['fruitset', 'fruitmass', 'seeds']]`:
   `df_berry3` 데이터프레임에서 `fruitset`, `fruitmass`, `seeds`
   열만 선택합니다.
2. `.apply(lambda x: x.clip(df_iqr.loc[x.name, 'LB'], df_iqr.loc[x.name, 'UB']))`:
   선택된 각 열에 대해 `apply` 함수를 사용하여 이상치를
   잘라내는 작업을 수행합니다.
'''
```

여기서 ``lambda x: x.clip(df_iqr.loc[x.name, 'LB'], df_iqr.loc[x.name, 'UB'])``는 각 열의 값을 상한선(UB)과 하한선(LB) 사이의 값으로 잘라내는 함수를 정의합니다.

``df_iqr.loc[x.name, 'LB']``는 해당 열의 이름을 통해 이상치의 하한선을 가져오고, ``df_iqr.loc[x.name, 'UB']``는 이상치의 상한선을 가져옵니다.

3. ``df_berry3[['fruitset', 'fruitmass', 'seeds']] = ...``:
이전 단계에서 수정된 값을 다시 ``df_berry3``의 해당 열에 할당하여 데이터프레임을 업데이트합니다.

따라서 이 코드는 이상치가 있는 값들을 해당 변수의 이상치 상한선과 하한선 사이의 값으로 대체하여 데이터를 정제하는 작업을 수행합니다.
...

```
df_berry3 = df_berry.copy()
# clip(하한, 상한)을 통해 상한과 하한으로 값을 제한합니다.
# apply에서 넘어온 Series의 name에는 해당 columns명이 들어 있습니다.
# 이를 통해 iqr에서 하한(LB), 상한(UB)를 가져오고
# pd.Series.clip을 통해 하한과 상한으로 값이 들어 가도록합니다.
df_berry3[['fruitset', 'fruitmass', 'seeds']]
    = df_berry3[['fruitset', 'fruitmass', 'seeds']].apply(
        lambda x: x.clip(df_iqr.loc[x.name, 'LB'], df_iqr.loc[x.name, 'UB'])
    )

df_berry3
```

Out[29]:

	clonesize	honeybee	bumbles	andrena	osmia	MaxOfUpperTRange	MinOfUpperTRange	AverageOfUpperTRange	MaxOfLowerTF
Row#									
0	37.5	0.750	0.250	0.250	0.250	86.0	52.0	71.9	
1	37.5	0.750	0.250	0.250	0.250	86.0	52.0	71.9	
2	37.5	0.750	0.250	0.250	0.250	94.6	57.2	79.0	
3	37.5	0.750	0.250	0.250	0.250	94.6	57.2	79.0	
4	37.5	0.750	0.250	0.250	0.250	86.0	52.0	71.9	
...	...	...	...	...	...	...	...	...	...
772	10.0	0.537	0.117	0.409	0.058	86.0	52.0	71.9	
773	40.0	0.537	0.117	0.409	0.058	86.0	52.0	71.9	
774	20.0	0.537	0.117	0.409	0.058	86.0	52.0	71.9	
775	20.0	0.537	0.117	0.409	0.058	89.0	39.0	65.6	
776	20.0	0.537	0.117	0.409	0.058	89.0	39.0	65.6	

777 rows × 17 columns



In []: